

Judge Demo Guide - IBM Bob Multi-Agent Workflow

A complete presentation script, command reference, and recovery plan for showcasing the project to judges.

Prepared for the IBM Bob regional hackathon demonstration

Duration: 7-10 minutes

Goal: Show a working API, then prove it was developed through a governed IBM Bob multi-agent workflow.

The one-sentence project pitch

We use IBM Bob custom modes to create specialized software-development agents with restricted permissions, independent review and testing, human-approved hand-offs, and an auditable evidence trail.

The to-do API is deliberately small. It is the working demonstration artifact; the main project is the controlled multi-agent development process.

Before the judges arrive

- Open two PowerShell terminals in the repository root:

```
cd "C:\Users\Test\Documents\Multi-Agent"
```

- In both terminals, activate the virtual environment:

```
.\.venv\Scripts\Activate.ps1
```

If the environment does not yet exist, create it once:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r .\app\requirements.txt
```

- Open these files in an editor before the presentation:
- .bob/custom_modes.yaml
- workflow/handoff_controller.py
- review_report.md
- test_report.md
- debug_report.md
- docs/workflow/handoff-log.jsonl
- Keep **Terminal 1** for the API server. Keep **Terminal 2** for workflow commands and tests.

Presenter note: Do not claim that IBM Bob automatically switches modes. A team member selects the next mode. The controller validates evidence and records the human-approved hand-off.

Step 1 - Opening: state the problem (45 seconds)

Sav

AI can generate code quickly, but one general AI agent may misunderstand requirements, miss security issues, write weak tests for its own code, or give an unsupported claim of completion. Our project introduces checks and balances: separate IBM Bob agents plan, implement, review, test, diagnose, and document the work.

Each agent has a limited role and restricted file access. Every stage leaves evidence for the next one.

Show

Open README.md and point to the workflow:

Research -> Coding -> Review -> Testing -> Debug or Docs

Sav

The FastAPI to-do application is intentionally simple, so the judges can focus on the multi-agent workflow rather than on unnecessary product complexity.

Step 2 - Show the IBM Bob custom modes (1 minute)

Show

Open .bob/custom_modes.yaml.

Point out the six agents:

Research Agent
Coding Agent
Review Agent
Testing Agent
Debug Agent
Pitch/Docs Agent

Sav

These are actual IBM Bob custom modes. They are not just six prompts written in a document. Each one has a role definition, detailed instructions, allowed capabilities, and edit permissions.

Show

Under Coding Agent, show:

fileRegex: "^app/.*"

Sav

Coding Agent can modify only files inside app/. It cannot alter the review report, test report, or audit history.

Show

Under Testing Agent, show:

fileRegex: "(app/test_.*\\.py|test_report\\.md)\$"

Sav

Testing Agent can write tests and its report, but it cannot modify the production API source. This prevents an agent from silently fixing its own failure.

Step 3 - Show the governed hand-off system (1 minute)

Run in Terminal 2

```
python workflow\handoff_controller.py status
Get-Content docs\workflow\status.md
```

Sav

The hand-off controller checks whether the required evidence exists before the workflow can move forward. For example, testing can begin only after an approved review, and documentation can begin only after a passing test report.

Optional live hand-off

This appends a real audit entry. Use your actual presenter name:

```
python workflow\handoff_controller.py handoff review testing --approved-by "Your Name"
```

Sav

The controller accepted this hand-off only because the review report is approved. It records the approver, evidence, next IBM Bob mode, expected inputs, and expected output.

Show

```
Get-Content docs\workflow\handoff-log.jsonl
```

Sav

This is our evidence trail. In a production version, we would add signed approvals, protected branches, centralized audit storage, and identity-based access control.

Step 4 - Show the working API (2 minutes)

Run in Terminal 1

```
python -m uvicorn app.main:app --reload
```

Open this URL in a browser:

```
http://127.0.0.1:8000/docs
```

Sav

This is FastAPI's automatically generated Swagger UI. It proves the server is running and shows the API contract: request formats, response schemas, status codes, and interactive calls for all six endpoints.

Demonstrate in Swagger UI

- Expand POST /todos.
- Click **Try it out**.
- Enter:

```
{
  "title": "Prepare IBM Bob demo"
}
```
- Click **Execute**.
- Copy the returned id.
- Run GET /todos and show the new item.
- Run PATCH /todos/{id}/toggle, entering the copied ID.

Sav

The item is persisted in SQLite. The toggle route changes done from false to true. FastAPI validates the input and exposes the API specification automatically.

Optional terminal equivalent

In Terminal 2:

```
$created = Invoke-RestMethod -Uri "http://127.0.0.1:8000/todos" -Method Post -ContentType "application/json" -Body '{
$created | ConvertTo-Json
$todoId = $created.id
Invoke-RestMethod -Uri "http://127.0.0.1:8000/todos/$todoId/toggle" -Method Patch | ConvertTo-Json
```

Step 5 - Show a real review finding (1 minute)

Show

Open `review_report.md`, then `app/main.py`.

Sav

The Review Agent found a dynamic SQL pattern in the update route. It was not directly exploitable in the current version because the field names were internally fixed, but it was a risky pattern that could become an injection vulnerability after a future change.

Show

In `app/main.py`, point to fixed static statements such as:

```
"UPDATE todos SET title = ?, done = ? WHERE id = ?"
```

Sav

We replaced dynamic construction with fixed SQL statements and parameterized values. Review Agent also caught a scope-document drift and inconsistent blank-title validation. This is evidence that the workflow found real issues, not only happy-path results.

Step 6 - Show independent testing (1 minute)

Run in Terminal 2

```
$env:PYTEST_DISABLE_PLUGIN_AUTOLOAD = "1"
python -m pytest app\test_main.py workflow\test_handoff_controller.py -q
```

Expected result

26 passed

Sav

The repository contains 23 independent API tests and 3 workflow-controller tests. Every test gets a fresh temporary SQLite database, so it never changes the real demo database.

Show

Open `test_report.md`.

Sav

The tests include happy paths, missing and blank titles, title-length boundaries, strict Boolean validation, invalid IDs, 404 responses, no-op updates, double deletion, and toggling in both directions.

We disable third-party pytest auto-loading only in this local Windows environment because an unrelated globally installed xonsh plugin requires an interactive console. This does not change the project test code; a clean virtual environment avoids that local plugin issue.

Step 7 - Optional failure-and-recovery demonstration (2-3 minutes)

Use this only if you have enough time. Do **not** commit or push the deliberately broken code.

A. Stop the API

In Terminal 1, press Ctrl + C.

B. Confirm the source file is safe to edit

```
git diff -- app\main.py
```

Proceed only if this shows no existing changes.

C. Introduce the controlled bug

Open:

```
notepad app\main.py
```

In the DELETE /todos/{id} route, change these two values:

```
status.HTTP_204_NO_CONTENT
```

to:

```
status.HTTP_200_OK
```

Save the file.

D. Show the failing test

```
$env:PYTEST_DISABLE_PLUGIN_AUTOLOAD = "1"
python -m pytest app\test_main.py::test_delete_todo -q
```

Sav

We intentionally changed the DELETE response contract. The operation still deletes the todo, but it incorrectly returns HTTP 200 instead of the required HTTP 204. The test catches that mismatch.

E. Run the Debug Agent in IBM Bob

Select **Debug Agent** mode and paste:

```
Read design_brief.md, test_report.md, app/test_main.py, and app/main.py.
```

The DELETE endpoint test fails because it expected HTTP 204 but received HTTP 200.

Do not modify application code. Update debug_report.md with the failed test, root cause, classification, recommended

Sav

Debug Agent does not fix code. It finds the actual root cause and gives Coding Agent a precise, minimal correction.

F. Fix through Coding Agent

Select **Coding Agent** mode and paste:

Read debug_report.md and fix only the reported DELETE endpoint issue in app/main.py.

The DELETE route returns HTTP 200, but the specification requires HTTP 204 No Content. Do not modify unrelated code.

G. Verify recovery

Run:

```
python -m pytest app\test_main.py::test_delete_todo -q
python -m pytest app\test_main.py workflow\test_handoff_controller.py -q
```

Sav

The test now passes again. This demonstrates the recovery branch: Testing detects the failure, Debug diagnoses it, Coding fixes only the diagnosed issue, and Testing verifies the repair.

After the repair, ask Testing Agent to update test_report.md back to PASS. Preserve the original Debug Agent diagnosis and append a resolution-verification note to debug_report.md.

Step 8 - Closing statement (30 seconds)

Sav

Our project does not claim that AI should replace engineering controls. It demonstrates that IBM Bob can support a safer process: specialized roles, least-privilege permissions, independent review and testing, explicit human approval, and traceable evidence.

Today this is a working proof of concept. The next step is integration with pull requests, enterprise identity, signed audit records, security scanning, and deployment workflows.

Fast 3-minute version

If the judges have little time:

- Open Swagger at <http://127.0.0.1:8000/docs>; create and toggle one todo.
- Open .bob/custom_modes.yaml; show Coding and Testing permissions.
- Run:

```
python workflow\handoff_controller.py status
```

- Show review_report.md; explain the SQL-pattern finding.
- Run:

```
$env:PYTEST_DISABLE_PLUGIN_AUTOLOAD = "1"
python -m pytest app\test_main.py workflow\test_handoff_controller.py -q
```

- Close with the project pitch above.

Judge questions to prepare for

Question	Short answer
Why not one AI agent?	A single agent can write, test, and approve its own work. Specialization adds independent checks.
Is IBM Bob fully automated here?	Bob provides the agents and permission boundaries; the human selects modes and approves hand-offs.
Why SQLite?	It makes the demo local, reproducible, and dependency-light. Production would use a managed database such as PostgreSQL.
Is this production-ready?	The workflow principle is enterprise-ready; the demo application intentionally omits auth, deployment, observability, and production infrastructure.
What real value did agents add?	Review Agent caught a risky SQL pattern, validation inconsistency, and scope drift; Debug Agent precisely diagnosed a failed DELETE response contract.
What is next?	Pull-request gates, identity-based approvals, immutable audit logs, a Security Agent, and enterprise integrations.